

Attacks on Vision-Language-Action Robotic Systems: Mechanisms, Evaluation, and Open Problems

Anonymous Author(s)

Abstract—Vision-Language-Action (VLA) models connect open-ended language and visual context to robot action, memory, tools, and future observations. This makes attacks qualitatively different from adversarial image classification, text-only jailbreaking, or conventional robot robustness: an intervention may shift object grounding, redirect a trajectory, trigger an action segment, leak training membership, or persist until recovery fails. This mechanism-driven survey reviews how VLA attack research has moved from adversarial vision, prompt injection, and robot-policy attacks toward direct attacks on VLA perception, instruction following, action generation, reasoning, and privacy. We organize the literature around a closed-loop attack map that tracks how adversarial effects enter, what they change inside the robot stack, whether they cross the controller boundary, and whether feedback leads to recovery or violation. To calibrate claims, we distinguish direct VLA evidence from embodied-adjacent and contextual work, and we formalize typed attack success as a rollout-level predicate tied to attacker assumptions, benign baselines, validation tier, and closed-loop consequence. Current direct VLA studies already show that action-producing and reasoning-enabled policies can be attacked, but deployment-level claims remain narrow: physical recovery studies are limited, privacy metrics are still emerging, and adaptive evaluation against guardrails is immature. We close with a robotics-oriented roadmap for physically grounded, privacy-aware, and recovery-aware VLA attack studies.

Index Terms—Vision-Language-Action Attacks, Robot Foundation Models, Survey, Embodied Security, Physical Attack Evaluation.

I. INTRODUCTION

Vision-Language-Action (VLA) models and robot foundation models are changing the interface between perception, language, and control. Instead of a narrow module that classifies an image or emits a low-level action under a fixed task, a VLA system can use open-ended language and visual context to condition robot action, memory, tool use, and future observations. Recent VLA/RFM systems demonstrate this shift across multimodal pretraining, robot-data scaling, cross-embodiment transfer, and natural-language task specification [1]–[8]. Related planner and policy systems show the same pattern at different interfaces: language and vision can steer skill selection, code generation, waypoint synthesis, trajectory distributions, and action-chunk prediction [9]–[15]. For robotics, this is not only a new model family. It is a new control interface whose inputs are semantically rich and whose outputs can move robots, call tools, update memory, and alter the future observation stream.

This interface changes what an attack means. In a VLM, an adversarial image may produce a wrong answer; in a

text-only LLM, a jailbreak may produce unsafe text; in a conventional robot policy, a disturbance may reduce reward or tracking accuracy. In a VLA-enabled robot, the same kinds of interventions can enter a closed loop. A visual patch can change grounding, the changed grounding can alter an action token or waypoint, the controller can clip or smooth the command, and the robot’s next observation can either recover the task or reinforce the deviation. A scene-text instruction can cross a source boundary, a poisoned demonstration can implant a trigger, a black-box prompt edit can redirect a trajectory, and an action-level backdoor can freeze or release a gripper at a critical moment. The consequence may be a wrong-object action, unsafe motion, unauthorized skill use, privacy leakage, or persistent state compromise, not just a misclassification or unsafe sentence.

A. Related Surveys and Positioning

Existing survey traditions do not fully capture this closed-loop attack problem. VLA/RFM surveys explain model architectures, datasets, action representations, and applications. Embodied-security and robot-cybersecurity surveys explain sensors, middleware, physical systems, and human-robot interaction. LLM-agent security surveys explain prompt injection, tool misuse, retrieval poisoning, and data leakage. The closest adjacent work is the VLA safety survey *Vision-Language-Action Safety: Threats, Challenges, Evaluations, and Mechanisms* [16], which covers the broader safety landscape: threats, alignment, evaluation, defense mechanisms, and deployment challenges. This paper asks a narrower attack-centered question: *how do adversarial interventions propagate through VLA-controlled robots, what mechanisms have been demonstrated, and when is the evidence strong enough to support a robotics claim?*

The distinction is increasingly practical as the literature moves from position statements to concrete VLA attack papers. Direct evidence now includes adversarial prompt and patch attacks on VLA policies [17]–[20], action freezing and action/trajectory backdoors [21]–[26], transferable and model-agnostic patch attacks [27], [28], 3D object-texture attacks [29], CoT-reasoning hijacking [30], trajectory redirection [31], and VLA membership inference [32], [33]. These papers are not just examples inside a safety taxonomy; they form the attack evidence that this survey compares.

TABLE I
HOW THIS SURVEY DIFFERS FROM ADJACENT SURVEY TRADITIONS. THE COMPARISON CONCERNS SCOPE AND EVIDENCE USE, NOT THE VALUE OF EACH LITERATURE.

Dimension	Closest VLA safety survey	Embodied security survey	LLM-agent security survey	This survey
Direct VLA attack evidence	Discusses VLA threats within broader safety.	Usually sparse or absent.	Usually absent.	Centers demonstrated attacks on VLA policies, VLA-controlled robots, and VLA training/privacy channels.
Evidence use	Threat/evaluation oriented.	Vulnerability oriented.	Software-agent risk oriented.	Separates direct VLA evidence from embodied-adjacent and contextual work so adjacent risks do not inflate VLA claims.
Action representation	Often covered as model capability.	Usually not central.	Not applicable.	Compares action token, chunk, waypoint, trajectory, velocity/flow, skill/API, controller, and memory representations.
Controller/recovery	Defense or safety mechanism discussion.	Cyber-physical component discussion.	Not applicable.	Highlights why chunk horizon, controller frequency, safety filters, recovery, and pre/post-controller traces change attack claims.
Privacy attacks	Usually safety-adjacent.	Sometimes privacy/security.	Often data leakage.	Treats VLA membership inference and VLALeaks as direct VLA confidentiality evidence.
Evaluation emphasis	Broad trend synthesis.	Broad vulnerability synthesis.	Benchmark scores or attack rates.	Interprets typed attack success, validation tier, controller boundary, transfer, privacy metrics, and recovery rather than pooling incompatible ASR.

B. Guiding Questions

We use five questions to keep the survey focused on mechanisms and robotic consequences.

- **RQ1:** Where do VLA attacks enter the robot system: vision, language, memory, tools, environment, action representations, or humans?
- **RQ2:** Which internal objects do attacks change: grounding, plans, reasoning traces, action tokens, chunks, trajectories, controller inputs, memory entries, or tool calls?
- **RQ3:** How do these changes propagate through execution, feedback, recovery, and future observations?
- **RQ4:** Which consequences have been demonstrated: task failure, wrong-object action, unsafe motion, privacy leakage, unauthorized skill use, or persistent compromise?
- **RQ5:** What evidence is strong enough for robotics conclusions, and where do physical validation, transfer, controller interaction, and recovery remain weak?

Table I compares this survey against the closest adjacent survey traditions. The difference is not simply that we are “attack-centered.” We ask how attack mechanisms interact with VLA action representations, closed-loop execution, controller boundaries, and the strength of the available evidence.

C. Scope and Definition

We treat a *VLA attack* as an intentional intervention that changes a VLA-enabled system’s perception, grounding, planning, policy decoding, execution, memory, tool use, privacy boundary, or human-robot interaction in a way that violates the intended task, authorization boundary, privacy expectation, or safety envelope. This definition separates adversarial attacks from ordinary task failure and non-adversarial robustness failure. A robot that fails because of unmodeled friction exhibits robustness risk; a robot that fails because an adversary selected a perturbation, instruction, tool response, memory

entry, trigger, or physical scene configuration to induce a violation exhibits security risk.

D. Evidence Use

We use a compact evidence review to calibrate claims, while organizing the survey around attack mechanisms and robotic consequences. Searches were last updated on 16 June 2026 and covered major robotics, machine learning, computer vision, security, and NLP venues, together with arXiv/OpenReview and linked project pages.

The survey distinguishes direct VLA evidence from embodied-adjacent and contextual work. *Direct VLA evidence* requires a VLA target and a VLA-relevant outcome, such as changed action, trajectory, reasoning-to-action behavior, triggered policy behavior, or training-data membership leakage. *Embodied-adjacent evidence* motivates risks around embodied planners, robot policies, sensing paths, middleware, and HRI, but still requires VLA-specific validation before it can support a VLA vulnerability claim. *Contextual evidence* explains substrates, benchmarks, adversarial methods, robot-security mechanisms, or action representations. Supplementary materials provide the search templates and evidence appendix. The main text uses this evidence base to support mechanism-level conclusions, not to pool heterogeneous ASR values.

E. Contributions

This paper discusses attacks at a responsible academic level and does not provide operational payloads, exploit recipes, or instructions for real-world misuse. It makes six contributions.

- We define the VLA attack problem as a closed-loop robotics security problem, where adversarial effects can propagate from perception, language, data, memory, tools, or middleware into robot actions, physical state, and recovery behavior.

- We formalize typed VLA attack success, separating task failure, targeted behavior, safety violation, authorization violation, privacy leakage, persistence, and recovery failure.
- We introduce an orthogonal attack map covering lifecycle, entry surface, target component, target representation, security property, attacker assumptions, validation tier, and consequence.
- We synthesize direct VLA attacks through their robotic mechanisms, with particular attention to action tokens, chunks, waypoints, trajectories, diffusion/flow policies, controller clipping, and recovery.
- We compare the strength and limits of current evidence, separating direct VLA attacks from embodied-adjacent attacks and contextual references without letting background work inflate attack claims.
- We propose evaluation principles for VLA attacks that connect typed ASR, benign utility, transfer, query budget, physical validation, closed-loop recovery, and consequence severity.

II. CLOSED-LOOP PROPAGATION OF VLA ATTACKS

A. Closed-Loop VLA System

A VLA-enabled robot should be read as a closed-loop state transition system, not as a single prompt-to-action function. Let x_t denote the physical state, $o_t = O(x_t, \eta_t)$ the observation produced by sensors and preprocessing noise η_t , m_t the memory/tool/human context, and u_t the command emitted by the VLA or its planner. A low-level controller K maps u_t and robot state to actuation, while a recovery or safety mechanism R may stop, replan, filter, or ask for help:

$$\begin{aligned} u_t &= \pi_\theta(o_{\leq t}, g, m_t), \\ a_t &= K(u_t, x_t, R_t), \\ x_{t+1} &= F(x_t, a_t, w_t), \quad m_{t+1} = U(m_t, o_t, u_t, a_t). \end{aligned} \quad (1)$$

The safety envelope is a set $\mathcal{S}$ over physical state, controller state, tool authorization, privacy state, and human proximity. An attack is relevant when an intervention δ changes one of these transitions or contexts so that the rollout violates a task, authorization, privacy, or safety predicate. This view gives the survey its central question: where does the adversary enter, which representation changes first, how does that change cross the controller boundary, and does feedback or recovery stop it?

Figure 1 is the organizing framework for the rest of the survey. It separates where the adversary enters from what the attack changes, marks the controller boundary, and summarizes the propagation chain

$$\begin{aligned} \delta &\rightarrow \Delta o_t \rightarrow \Delta z_t \rightarrow \Delta u_t \rightarrow \Delta a_t \\ &\rightarrow \Delta x_{t+1} \rightarrow \text{recovery or violation} \end{aligned} \quad (2)$$

The same perturbation can stop at the observation, change an embedding or grounding representation, alter a plan, shift an action token, corrupt a chunk, redirect a trajectory, be clipped by a controller, or only become visible after the next observation. This is why a VLA attack claim is not settled

at the model-output boundary; the robot-level claim depends on execution, feedback, recovery, and the resulting physical or system consequence.

B. Attack Map

The attack map has four intuitive stages. First, the attack enters through a channel: vision, language, memory, tool output, middleware, training data, physical environment, or human interaction. Second, it changes a target representation: grounding, plan, reasoning trace, action token, action chunk, waypoint, trajectory, velocity field, controller state, memory entry, or tool call. Third, the changed representation reaches a robot or system consequence: task failure, wrong-object action, unsafe motion, unauthorized skill use, privacy leakage, or persistent state compromise. Fourth, the validation setting determines what the claim can support: digital input, real-sensor input, simulation rollout, hardware-in-the-loop, real-robot actuation, or human-proximate study.

Table II keeps these stages orthogonal because common labels hide different mechanisms. “Physical attack” describes an access channel, not the target: a physical patch may attack visual embeddings, grounding, visual proprioception, or a trajectory objective. “Jailbreak” describes a family resemblance, not a consequence: a prompt attack may cause task failure, targeted motion, unauthorized tool use, or no robot-level effect. “Backdoor” describes timing and training access, but the robotic target may be a gripper primitive, an action chunk, a waypoint, or a long-horizon trajectory.

C. Typed Attack Success

Because VLA safety papers do not yet use a uniform definition of attack success, a survey needs a typed definition. Let $\tau^0(c)$ be the benign rollout from an initial condition and task context $c = (x_0, g, m_0)$, and let $\tau^\delta(c)$ be the rollout under an adversarial intervention δ that satisfies the declared knowledge, access, privilege, budget, and physical constraints. A VLA attack is successful for objective type O and consequence class C when

$$\begin{aligned} \text{Succ}_{O,C}(\tau^\delta, \tau^0) &= \mathbf{1}\{\phi_{O,C}(\tau^\delta) = 1 \\ &\quad \wedge \phi_{O,C}(\tau^0) = 0 \mid \delta \in \Delta(\mathcal{A})\}. \end{aligned} \quad (3)$$

where $\phi_{O,C}$ is a typed violation predicate and $\Delta(\mathcal{A})$ is the admissible attacker set. The benign rollout condition prevents ordinary task difficulty or non-adversarial distribution shift from being counted as attack success. If a benign rollout is unavailable, the paper should state the baseline used, such as clean task success, human-labeled intended behavior, or non-member trajectory statistics.

The predicate $\phi_{O,C}$ must be typed. A *task-failure attack* succeeds when the benign task would complete but the attacked rollout fails. A *targeted behavior attack* succeeds when the robot reaches an attacker-specified object, action primitive, waypoint, trajectory, skill, or tool call. A *safety attack* succeeds when the rollout violates a safety envelope, contact margin, force/velocity/workspace bound, or human-proximity constraint. An *authorization attack* succeeds when

TABLE II
ORTHOGONAL ATTACK-MAP DIMENSIONS FOR VLA ATTACKS. EACH PAPER MAY TOUCH MULTIPLE DIMENSIONS; THE DIMENSIONS ARE NOT MUTUALLY EXCLUSIVE CATEGORIES.

Axis	Common labels	Guiding question
Lifecycle	Training; inference; runtime; post-deployment	When is the adversarial intervention introduced or activated?
Entry surface	Vision; language; state; memory; tool; middleware; human; environment	Which semantic or system channel carries the adversarial content?
Target component	Visual encoder; language encoder; multimodal projector; planner/reasoner; action head; controller; memory; tool/API; middleware	Which system component is attacked or measured?
Target representation	Embedding; grounding; plan; action token; chunk; waypoint; trajectory; velocity field; controller state; memory entry	Which representation or control object changes inside or across components?
Security property violated	Availability; integrity; confidentiality; authorization	Which security property is violated? Physical safety is represented as a consequence rather than merged into this axis.
Knowledge level	White-box; gray-box; black-box; transfer-only; unknown	What does the attacker know about weights, gradients, architecture, prompts, action heads, or controllers?
Access channel	Digital; physical; human-mediated; supply-chain; hybrid	How does the attacker deliver the intervention? This is separate from knowledge level.
Privilege level	User; bystander; operator; data contributor; model provider; tool/API; middleware; no privilege	What role or privilege boundary does the attacker occupy or abuse?
Temporal profile	One-shot; repeated; delayed; persistent	Does the attack act once, recur over feedback, activate later, or survive sessions/training?
Validation tier	Digital input; real sensor; simulation; HIL; robot actuation; human-proximate	What is the strongest demonstrated validation tier? Do not infer actuation or human proximity from input-only tests.
Consequence	Task failure; wrong object; unsafe action; collision/contact risk; privacy leakage; unauthorized skill; persistent state compromise	What happened at the robot/system boundary? Keep consequence separate from objective and entry surface.

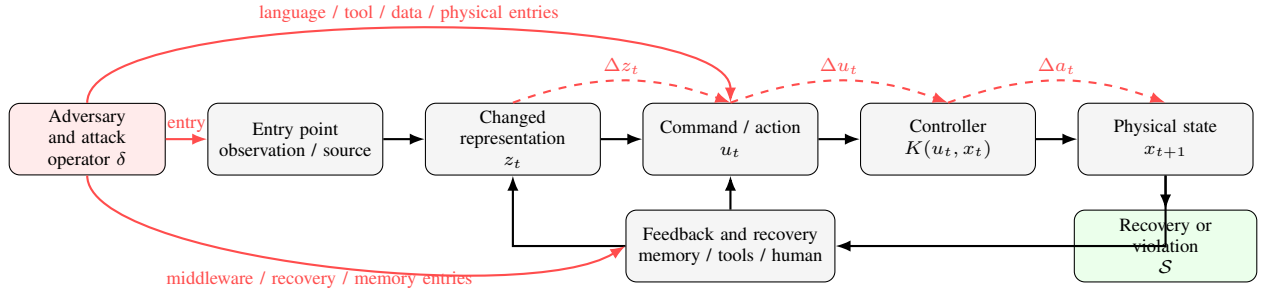


Fig. 1. Core closed-loop view used throughout the survey. The figure separates adversarial entry surfaces from representation impact, marks the controller boundary between model command and physical actuation, and shows how feedback either supports recovery or carries the system toward a robot/system violation.

the system executes an unauthorized command, skill, tool, or privilege transition. A *privacy attack* succeeds when membership, trajectory, user, or scene information is inferred above a stated threshold. A *persistence attack* succeeds when the adversarial effect survives replanning, feedback, memory update, fine-tuning, or a session boundary. A *recovery-failure attack* succeeds when the system has an opportunity to stop, replan, ask for help, or invoke a recovery policy but still reaches the violation predicate.

This definition makes ASR meaningful only when five items are stated together: the violation predicate, the denominator, the attacker assumptions, the validation tier, and the benign or clean baseline. Without these items, ASR values from different VLA papers should be treated as local evidence rather than comparable field-level rates.

D. Attacker Assumptions

We assume the deployer has an intended task, authorized users, authorized tools and skills, and a safety envelope for people, property, data, and robot hardware. The attacker seeks

to violate one or more of these constraints while preserving enough surface plausibility that the system continues acting, logs the event as normal, or stores corrupted context for future sessions.

Attacker capability has three independent dimensions. *Knowledge level* describes what the attacker knows about weights, gradients, architecture, prompts, action heads, or controllers: white-box, gray-box, black-box, transfer-only, or unknown. *Access channel* describes how the intervention is delivered: digital, physical, human-mediated, supply-chain, or hybrid. *Privilege level* describes the role or boundary involved: user, bystander, operator, data contributor, model provider, tool/API, middleware, or no privilege. Keeping these dimensions separate prevents a common confusion: “black-box” and “physical” are not alternatives. A physical patch can be optimized with white-box gradients, transferred in a black-box setting, or deployed by a bystander with no digital privileges.

E. Reading the Map Through Examples

The map is easiest to read through examples. In a scene-text hijacking scenario, the attacker does not need to rewrite the user’s goal; the robot sees an instruction-like phrase in the environment, the phrase changes source grounding or plan selection, and the downstream consequence may be wrong-object manipulation or unauthorized skill use. In a patch scenario, the entry is visual, but the relevant question is whether the perturbation merely changes an embedding or whether it reaches an action token, waypoint, or physical trajectory [18], [27], [28]. In a poisoned-demonstration scenario, the attack enters before deployment, hides behind clean utility, and later activates as a gripper primitive, action freeze, or chunk-level drift [21], [24], [26]. In a trajectory-redirection scenario, the language may remain close to the user’s instruction while the physical path bends toward a different endpoint [31]. In a membership-inference scenario, the consequence is not motion failure at all; action likelihoods or temporal action traces leak whether a demonstration or trajectory was in the training set [32], [33].

This separation also prevents overclaiming. A VLM jailbreak with no robot action is contextual evidence for source confusion, not direct evidence of VLA unsafe action. A ROS2 middleware exploit is embodied-adjacent evidence for tool/middleware compromise, not a VLA policy attack unless the VLA system consumes or emits the compromised message. A physical LiDAR or camera attack can be strong sensor evidence but remains contextual for VLA action unless a VLA-controlled robot rollout is measured.

III. EVOLUTION OF VLA ATTACK MECHANISMS

The field has developed in waves. The first wave came from adversarial vision and physical patches: if camera inputs can be perturbed, robot perception and grounding may change. The second wave came from prompt injection, embodied jailbreaks, and robot-policy attacks: language and task context can steer plans, skills, and sequential decisions. The current wave attacks VLA systems directly: patches are optimized for action deviation, prompts are evaluated by rollout behavior, backdoors target primitives and chunks, reasoning traces are treated as control-relevant state, trajectories are redirected, and action traces become privacy signals. This section follows that evolution by mechanism rather than by modality alone.

A. Training-Time, Supply-Chain, and Backdoor Attacks

Training-time attacks arise because VLA systems are adapted from large robot datasets, demonstrations, and fine-tuning corpora. A poisoned demonstration set may look normal during training while associating a trigger with a gripper primitive or a small delta-pose drift. At deployment, the robot can still perform clean tasks, yet the trigger causes a release, freeze, wrong-object action, or gradual trajectory shift.

VLA backdoor work has therefore moved beyond generic task disruption. The common mechanism is trigger-conditioned control of a robot-relevant representation while clean behavior remains plausible. Some studies target triggered action deviations or physical-object-conditioned goals [22],

[23]. Others bind triggers to reusable primitives, long-horizon sequences, static action states, or chunk-level drift [21], [24]–[26]. The target is no longer a label; it is the command object the robot will execute.

The robotic issue is temporal control. A backdoor that fires a primitive at the wrong decision window differs from one that slowly biases a chunk, and both differ from a trigger that changes a high-level goal. Direct VLA studies are stronger than classifier analogies because they measure action behavior and often clean-task retention, but the setting remains narrow: most results use curated manipulation suites, controlled triggers, and limited embodiments. Current evidence leaves unresolved whether triggers survive data filtering, downstream fine-tuning, tokenized versus continuous action heads, chunked policies, flow/diffusion policies, and real controller clipping. Classic poisoning/backdoor work [55], [56], [58] and RL backdoors [53], [60] explain the supply-chain mechanism, but direct VLA work is what makes the robotics claim.

B. Perception, Multimodal, and Physical Attacks

Visual and physical attacks are the most natural bridge from adversarial ML to VLA robotics, but fooling an image model is not enough. The robotics claim begins when a perturbation survives the perception-to-action chain: a patch or object texture changes which object the robot grounds, shifts the action trajectory, and then either gets corrected by feedback or reaches the controller as a wrong-object motion.

Early VLA patch work made this shift explicit by optimizing action discrepancy, position-aware objectives, and targeted manipulation behavior rather than image classification error [18]. Other work attacks earlier representations: VLA-Fool studies textual, visual, and cross-modal misalignment, while EDPA and ADVLA target visual embeddings, projected features, attention, and vision–language alignment [19], [27], [35]. Transfer-oriented work then asks whether shared feature-space, attention, semantic, or visual-proprioception structure can carry attacks beyond one model-specific action head [28], [36]. Physical realism appears in object-bound texture attacks and in partially observable settings where the adversary sees only a short trajectory prefix [29], [37].

The mechanism sequence is now clearer. Action-space patches ask whether visual perturbations can change robot motion. Embedding and attention attacks ask whether the shared multimodal representation is enough when the action head is unknown. Transferable patches ask whether the attack survives architecture, fine-tuning, viewpoint, and sim-to-real changes. 3D texture attacks ask whether the manipulated object can carry the perturbation. Partial-observation attacks ask whether the attack can be planned without knowing the full future rollout.

The unresolved issue is not whether vision can be perturbed; adversarial examples and physical patches have already established that background mechanism [48], [49], [62], [68], [69]. 3D, LiDAR, and audio-command settings extend the same concern beyond flat images [50], [74], [78]. What remains uncertain is how reliably a physical perturbation changes grounding, action, trajectory, and recovery across robot embodiments and controllers.

TABLE III

REPRESENTATIVE MECHANISM SYNTHESIS BY LITERATURE ROLE. CORE ROWS SUMMARIZE ATTACK MECHANISMS; CONTEXTUAL ROWS EXPLAIN SUBSTRATES OR EVALUATION CONTEXT BUT ARE NOT TREATED AS DIRECT VLA ATTACK RESULTS. D: DIGITAL; S: SIMULATION; HIL: HARDWARE-IN-THE-LOOP; P: PHYSICAL ROBOT/WORLD; PR: PEER-REVIEWED; PP: PREPRINT OR NON-ARCHIVAL VERSION.

Role	Representative work	Status	Validation	What the evidence supports
Direct VLA: instruction/control	RoboGCG, VLA-Fool, SABER, Q-DIG, trajectory redirection [17], [19], [20], [31], [34]	PP	D/S/P	Language or multimodal perturbations can alter VLA rollout behavior; black-box, natural-prompt, and command-preserving settings are emerging.
Direct VLA: patches/physical vision	ICCV patch attacks, EDPA, ADVLA, UPA-RFAS, VLA-Hijack, Tex3D, partially observable patches [18], [27]–[29], [35]–[37]	PR/PP	D/S/P	Visual attacks can target embeddings, grounding, visual proprioception, action trajectories, and object textures; transfer and physical evidence are improving but uneven.
Direct VLA: action/backdoor	FreezeVLA, BadVLA, GoBA, DropVLA/TabVLA, AttackVLA/BackdoorVLA, SilentDrift [21]–[26]	PP	S/P	VLA attacks can target action primitives, action freezing, chunk drift, and long-horizon triggered behavior, often while preserving clean utility.
Direct VLA: reasoning/privacy	TRAP, altered reasoning traces, ReasonBreak, VLA membership inference, VLALeaks [30], [32], [33], [38], [39]	PP	D/S/P	Reasoning traces and training membership are VLA-specific target representations; objectives include integrity and confidentiality rather than only task failure.
Core: embodied-adjacent attacks	BadRobot, RoboPAIR, CHAI, RoboJailBench, ANNIE, Phantom Menace, Eva-VLA [40]–[46]	PR/PP	D/S/P	Embodied LLM/VLM, environmental command, trajectory, sensor, and physical-variation attacks inform VLA threat models but are not counted as direct VLA evidence unless VLA policies are attacked.
Contextual references	VLA/RFM substrates, benchmarks, adversarial ML, physical/sensor attacks, RAG/tool-agent security, RL backdoors, ROS/robot cybersecurity [6], [8], [47]–[54]	PR/PP	D/S/P	Provides mechanisms, substrates, and evaluation lessons; not treated as attack evidence unless the work has explicit attacker, target, method, and outcome.

C. Instruction, Planning, and Reasoning Attacks

Instruction attacks matter because VLA systems treat language as a control input, not only as a conversation channel. In scene-text hijacking, the user asks for a normal task, but text in the environment or a small edit in the instruction changes object binding, subgoal order, or trajectory preference. A more subtle case is command-preserving redirection: the instruction remains close to the intended task while the physical path moves toward a different outcome.

RoboGCG adapts adversarial prompt optimization to VLA control authority and shows that a prompt-level intervention at rollout start can influence action reachability over time [17]. VLA-Fool includes textual and cross-modal misalignment in addition to visual perturbations [19]. SABER makes the instruction threat model more practical by using a black-box edit process with bounded character, token, or prompt changes and rollout-level feedback across multiple VLA models [20]. Q-DIG approaches red-teaming differently: it uses quality-diversity search to generate natural, task-relevant instructions that expose robot-policy failures and can be used for robustness improvement [34]. Trajectory-level redirection studies show why instruction similarity is not enough; a prompt can remain close to the intended command while redirecting the physical rollout [31].

Reasoning-enabled VLAs add another target between language and action. TRAP shows that CoT or intermediate reasoning traces can be hijacked by physical patches to steer downstream actions without changing the user’s instruction [30]. Altered-thought studies indicate that entity references in VLA reasoning traces can causally affect robot manipulation success, while non-reasoning control models may be immune to that specific channel [38]. ReasonBreak extends the same concern to autonomous-driving-style VLA systems, evaluating black-box textual corruptions against reasoning and trajectory safety metrics in closed-loop simulation [39]. In these architectures, reasoning text is not merely explanation; it can become

state that the controller ultimately acts on.

Embodied jailbreak and planning benchmarks provide useful context. BadRobot, RoboPAIR, and RoboJailBench evaluate embodied jailbreaks where success is measured closer to action than text-only refusal [40], [41], [43]. CHAI, SafeAgentBench, and EARBench add physical command hijacking and risk-aware planning settings [42], [82], [83]. Their limitation for this survey is target specificity: when the victim is an LLM-controlled planner rather than a VLA policy, the result motivates the attack surface but does not prove VLA action vulnerability. Direct VLA studies still need to test how instruction and reasoning attacks behave under prompt hierarchy, source labels, physical context changes, and closed-loop recovery.

D. Action-Level and Closed-Loop Attacks

Action representation is the robotics core of the survey. In an action-freezing attack, the perception and language inputs look plausible, but the policy repeatedly emits a static or blocked action state, so the robot stalls or fails to release control at the right moment. Chunk drift is subtler: each predicted delta-pose is only slightly biased, but the robot executes a multi-step chunk before new observations can correct the drift.

Autoregressive action-token policies expose token probabilities and discretized gripper or pose bins. A prompt or patch can change which token is selected, and early token errors can propagate through subsequent generation. RT-1, RT-2, OpenVLA, and SpatialVLA illustrate tokenized or discretized interfaces whose vocabularies, spatial grids, and detokenizers become attack-relevant objects [2], [3], [6], [84]. Parallel action chunks create a different risk: the model commits to several future actions before new observations can correct the error. ACT, OpenVLA-OFT, and related high-throughput fine-tuning recipes show why chunk length, temporal ensemble, and parallel decoding are security variables as well as effi-

ciency variables [15], [85]. SilentDrift directly exploits this intra-chunk open-loop property [26].

Waypoint and trajectory policies expose geometric objectives such as end-effector displacement, curvature, unsafe workspace entry, or targeted path redirection [18], [31]. Planner-mediated systems make object binding, generated code, value maps, and waypoint plans inspectable; CLIPort, Code as Policies, ProgPrompt, and VoxPoser show why these intermediate objects can become attack targets [10]–[13]. Diffusion and flow policies shift the target from a discrete token to a conditional trajectory distribution, denoising path, velocity field, or sampling process [8], [14]. Generalist policy work defines cross-embodiment assumptions [5], [7], [89]; robot-data collections expose provenance and embodiment choices that shape transfer claims [86]–[88].

Whether an action-level attack becomes physically consequential depends on controller and recovery design. A temporal ensemble can damp high-frequency perturbations but may also prolong a poisoned chunk. A high re-planning frequency can recover from one-frame perception attacks but remain vulnerable to repeated low-amplitude nudges. Controller latency and stale observations can turn a semantically correct command into an unsafe physical action. Safety envelopes and low-level controllers can prevent direct collision while still allowing wrong-object manipulation, unauthorized skill use, privacy leakage, or near-boundary behavior. The difficult case is not only attacking an action representation, but showing that the attack crosses the controller boundary and survives recovery.

E. System, Memory, Tool, HRI, and Persistent Attacks

VLA deployments will not be isolated policies. They will use tools, maps, object memories, user preferences, middleware, logs, and human instructions. A persistent-compromise case arises when a corrupted object memory or user preference is written during one session and retrieved in a later session, causing the robot to choose the wrong object or tool even though the new instruction is benign. Another case is an untrusted tool response that changes a skill argument or authorization boundary.

Tool-agent attacks and indirect prompt injection show how untrusted context can override source boundaries and cause tool misuse or data leakage [52], [90], [92]. Related benchmarks and attack studies sharpen the same source-confusion problem for software agents [91], [93]. In robotics, the analogous failure can become an unauthorized skill call, unsafe tool argument, or middleware command. Toolformer, ReAct, and ToolLLM are useful as software-agent substrates, while WebShop and WebArena illustrate interactive task settings [94], [95], [97]–[99]. VLA claims still require robot skills or VLA-controlled actions in the loop. ROS/ROS2 and robot cybersecurity work show that middleware, topics, services, node identity, and controller interfaces can be compromised [54], [100], [106]. Industrial, surgical, and service-robot studies show analogous weaknesses in deployed platforms [102]–[104]. HRI work shows that robots can affect human trust, authority, and physical access decisions [107], [109], [110].

Memory and privacy broaden the consequence space. RAG and memory poisoning show how retrieved context can be corrupted and reused later [51], [111]–[113]. For VLA robots, multi-session embodied memory compromise remains underdeveloped: poisoned maps, object memories, tool logs, or preferences may survive correction and later redirect behavior. VLA membership-inference work shows a different risk. Action likelihoods, action errors, temporal motion patterns, attention discrepancies, and trajectory structure can leak training membership [32], [33]. That is a direct VLA security problem even when no collision or task failure occurs; evaluation must connect confidentiality, persistence, authorization, and physical task performance.

IV. WHAT CURRENT EVIDENCE SHOWS

The direct VLA attack literature now supports mechanism-level conclusions, but not deployment-wide risk estimates. A useful comparison is therefore not a single leaderboard. It asks how the attack enters, which internal object changes, what robot behavior follows, and how far the validation setting carries the claim.

A. Mechanism-Level Patterns

Current work shows that VLA attacks are increasingly attacks on control-relevant internal objects, not only on inputs. Visual patches can move embeddings, object grounding, visual proprioception, and trajectory objectives. Instruction edits can steer plans, shift action tokens, or redirect a trajectory while preserving the apparent task. Backdoors can bind triggers to primitives, action freezing, or chunk-level drift. Privacy attacks use action likelihoods and temporal traces to infer training membership. A survey organized only by input modality misses these robotics-specific pathways.

A second pattern is that the strongest results demonstrate vulnerable VLA components rather than deployed hazard rates. Many papers show that an attack changes a rollout; fewer show whether the command is clipped, delayed, rejected by a safety filter, stopped by an emergency monitor, or corrected after a new observation. A patch that changes an action head in simulation, a prompt that changes a LIBERO rollout, and a backdoor that changes a gripper primitive are all important, but they do not imply the same physical risk.

The clearest limitation is the gap between attack realism and robotics-aware evaluation. Attack papers now include black-box instruction edits, transferable patches, object-bound 3D textures, partial-observation settings, and supply-chain triggers. Evaluation still lags on controller traces, HIL studies, recovery, human-proximate settings, and adaptive attacks against deployed guardrails. Stronger attacks without controller and recovery evidence do not answer deployment questions; evaluation without adaptive attacks can understate residual risk.

B. Representative Work by Mechanism

The current direct literature separates four mechanism families that are often conflated. Instruction-control attacks enter through language but are judged by robot behavior. RoboGCG

TABLE IV
REPRESENTATIVE DIRECT VLA ATTACK MECHANISMS AND WHAT EACH MECHANISM CURRENTLY SUPPORTS. THE TABLE IS QUALITATIVE: IT SUMMARIZES MECHANISM-LEVEL EVIDENCE RATHER THAN POOLING HETEROGENEOUS ASR VALUES.

Attack family	Representative entry	Changed representation	Robotic consequence	Evidence strength and caveat
Instruction/control	Prompt edit, scene text, or cross-modal instruction perturbation	Plan, source grounding, action token, or command-preserving trajectory	Task failure, action inflation, wrong-object behavior, or trajectory redirection	Direct rollout evidence is emerging; transfer, query budget, and recovery still vary across studies.
Visual/physical	Digital patch, printed patch, object texture, or partial-observation physical access	Embedding, grounding, visual proprioception, action loss, or trajectory objective	Wrong-object action, targeted motion, trajectory deviation, or degraded manipulation	Strong mechanism evidence across digital, simulation, and some real-robot settings; physical generality across viewpoints, objects, and controllers remains open.
Backdoor/action	Poisoned demonstration, trigger object, or supply-chain fine-tuning	Primitive, action freeze, chunk drift, waypoint, or long-horizon action sequence	Triggered release/freeze, wrong goal behavior, or gradual trajectory shift	Direct VLA studies show action representations are attackable while preserving clean utility; robustness to filtering, adaptation, and controller clipping is less mature.
Reasoning/privacy	Physical/contextual reasoning perturbation or action-trace query access	CoT/entity reference, reasoning-to-action state, likelihood, attention, or temporal trajectory signature	Reasoning hijack, unsafe driving/manipulation behavior, or training-membership leakage	Expands the field beyond task-failure ASR; privacy and reasoning claims require metrics different from physical task success.

studies whether adversarial text can create low-level control authority [17]; SABER uses bounded black-box edits to induce task failure, action inflation, or constraint violations [20]; Q-DIG uses natural prompt diversity to expose policy failures [34]; and trajectory redirection tests whether a command-preserving prompt can bend the rollout toward an attacker-specified outcome [31]. These attacks share an entry surface but differ in objective, attacker cost, and degree of targeting.

Visual attacks divide by where they act in the perception-to-action chain. Action-space attacks optimize VLA-specific action discrepancy and trajectory objectives [18]. Feature-level attacks move earlier, targeting visual embeddings, projected features, attention, or vision–language alignment [27], [35]. Transfer-oriented work exploits shared feature-space, semantic, attention, or visual-proprioception structure rather than a model-specific action head [28], [36]. Object-bound textures and partially observable patches add physical constraints: the perturbation must move with the object or be optimized from a short trajectory prefix [29], [37].

Action/backdoor attacks are not merely training-time versions of patches. They target the object that the policy will later execute: an action deviation, physical-object-triggered goal, reusable primitive, long-horizon sequence, static action state, or chunked delta-pose drift [21]–[26]. For comparison, the attacked action representation matters more than the trigger type alone.

Reasoning and privacy attacks expand the objective set. TRAP and altered-thought evaluations show that reasoning traces, entity references, and CoT-to-action pathways can become control-relevant attack targets [30], [38]. Reason-Break shows a related reasoning–trajectory vulnerability in autonomous-driving VLA systems, where collision and safety metrics matter more than tabletop task success [39]. Membership inference and VLALeaks show that action likelihoods, action errors, temporal motion patterns, attention discrepancies, and trajectory structure can leak training membership [32], [33]. These attacks require confidentiality metrics rather than

task-failure ASR.

C. Comparison Limits

ASR is not directly comparable across the current VLA attack literature. In one paper, ASR may mean task failure under a visual patch; in another, targeted gripper release within a reaction window; in another, action inflation; in another, trajectory redirection; and in privacy work, AUC or TPR@FPR for membership inference. The denominator also varies: image frames, task episodes, triggered episodes, candidate trajectories, model queries, or physical trials. Validation ranges from digital input-only to simulation rollout to real-robot actuation. Pooling these numbers would create false precision.

Comparable claims are therefore qualitative and typed. Direct VLA evidence supports the claim that VLA policies can be attacked through vision, language, action representations, backdoors, reasoning traces, and privacy channels. It does not yet support a universal claim that any particular attack transfers across all VLA architectures, embodiments, controllers, or safety envelopes. Embodied-adjacent evidence supports risk hypotheses about source confusion, tool misuse, middleware compromise, sensor corruption, and HRI authority ambiguity. Contextual evidence supports mechanisms and evaluation requirements, not VLA vulnerability claims by itself.

D. Strengths and Limits

The strongest direct evidence is concentrated in tabletop manipulation with a small set of open VLA targets and benchmarks. This creates a common baseline, but it weakens claims about general VLA security. Flow/diffusion policies, bimanual systems, mobile manipulation, navigation, tactile policies, and high-level tool/skill interfaces are less consistently covered. Action representations are also unevenly studied: tokenized action heads and patch-induced trajectory deviation are common; diffusion/flow denoising paths, velocity fields, controller

latency, temporal ensembling, recovery policies, and safety-envelope interactions are much less mature.

Physical validation is improving but remains narrow. Some direct VLA papers include physical robot demonstrations, but few provide enough detail on sensor stack, control frequency, calibration, lighting, object mass, latency, recovery behavior, and safety filters to make deployment claims portable. Hardware-in-the-loop is surprisingly underdeveloped as a middle tier between simulation and full physical trials.

Privacy is an emerging direct VLA attack area rather than a side issue. Membership inference papers show that VLA training data can leak through action outputs, likelihoods, attention patterns, and temporal motion signatures [32], [33]. These results support confidentiality conclusions, not physical-safety conclusions. Future work should therefore include privacy objectives rather than treating all VLA attacks as task-failure attacks.

E. Target Model and Benchmark Concentration

Target-model concentration is informative. OpenVLA and OpenVLA-OFT appear frequently because they are open, strong, and easy to evaluate on LIBERO [6], [47], [85]. π_0 and π_0 -fast matter because flow or diffusion-style action generation changes both inference speed and attack target representation [8]. SpatialVLA matters because spatial action grids and 3D spatial encoding create a different target representation from plain tokenized actions [84]. UniVLA, CronusVLA, reasoning VLAs, Alpamayo-style driving VLAs, and VLA-Adapter broaden the evidence base, but they are not yet as repeatedly tested as OpenVLA-style manipulation systems [26], [30], [36], [39].

The benchmark distribution is similarly concentrated. LIBERO is the dominant manipulation benchmark in the direct attack matrix, with RLBench, CALVIN, ManiSkill, RoboCasa, and DROID-derived data serving more often as capability or transfer context [86], [114]–[117]. This concentration is useful for comparability but can overrepresent tabletop manipulation, fixed cameras, short horizons, and simplified safety envelopes. Benchmark audits and robot-data papers therefore remain relevant to attack evidence even when they are not attacks themselves [5], [87], [88], [118].

F. Evaluation Weaknesses

Clean utility is usually clearer for backdoors than for inference-time attacks, because a backdoor must remain hidden under benign inputs. Transfer is most visible in patch and benchmark papers, but transfer axes differ: model transfer, task transfer, viewpoint transfer, suite transfer, and sim-to-real transfer are not interchangeable. Query budget appears in black-box instruction work but is absent from many visual and backdoor studies. Adaptive settings remain rare: most papers test a fixed attack against an undefended or lightly defended policy, not against a deployed prompt hierarchy, source-authentication layer, action shield, or runtime monitor. Recovery is the weakest dimension. Even when physical trials are included, many studies stop at task failure or target action induction rather than asking whether the robot could stop, replan, ask for help, or recover under controller constraints.

G. Open and Early Topics

Some topics are genuinely open: multi-session memory poisoning of VLA robots, adaptive attacks against deployed safety envelopes, HRI attacks with ethically controlled human subjects, and standardized HIL benchmarks. Other topics have preliminary direct evidence but insufficient breadth: transferable visual patches, black-box instruction attacks, action-level backdoors, reasoning-chain attacks, and VLA privacy attacks. These should be described as early but real VLA evidence, not as unexplored.

V. JUDGING VLA ATTACK CLAIMS

To judge a VLA attack claim, ask four questions. What representation changed? What robot or system consequence followed? What validation setting supports the claim? Does the result survive benign utility, transfer, controller interaction, and recovery? Table V summarizes the information needed to answer these questions, but the goal is interpretation rather than paperwork.

A. Typed Success Rather Than One ASR

Untyped ASR is insufficient because VLA attacks have different goals. A patch-induced task failure, a targeted gripper release, an action freeze, a redirected trajectory, an unauthorized skill call, and membership inference are not the same event. Typed rates can be derived from the success definition in Section II-C. Let $\kappa_O(\tau^\delta)$ indicate that adversarial rollout τ^δ satisfies objective type O , let $\sigma_C(\tau^\delta)$ indicate violation of consequence class C , and let $\rho_H(\tau^\delta)$ indicate that the attack effect persists after horizon H feedback steps, replans, memory updates, or sessions. For N trials,

$$\begin{aligned} \text{ASR}_O &= \frac{1}{N} \sum_{i=1}^N \mathbf{1}\{\kappa_O(\tau_i^\delta) = 1\}, \\ \text{SVR}_C &= \frac{1}{N} \sum_{i=1}^N \mathbf{1}\{\sigma_C(\tau_i^\delta) = 1\}, \\ \text{PR}_H &= \frac{1}{N} \sum_{i=1}^N \mathbf{1}\{\rho_H(\tau_i^\delta) = 1\}. \end{aligned} \quad (4)$$

Here ASR_O may refer to untargeted failure, targeted object choice, gripper primitive induction, action freezing, trajectory redirection, privacy membership inference, unauthorized skill invocation, or persistent memory compromise. SVR_C measures consequence-specific violations such as unsafe action, collision/contact risk, privacy leakage, or authorization failure. PR_H is required for backdoors, memory attacks, repeated instruction edits, action-chunk drift, and attacks that exploit recovery behavior.

For robot deployments, success should also be conditioned on the controller and recovery system. Let $r(\tau^\delta)$ indicate that a recovery opportunity occurred, $q(\tau^\delta)$ that the controller or safety filter rejected the command, d_{pre} and d_{post} be pre- and post-controller deviations from the benign action or trajectory,

TABLE V
MINIMUM DETAILS NEEDED TO INTERPRET VLA ATTACK CLAIMS. THE TABLE KEEPS THE FOCUS ON WHAT WAS ATTACKED, HOW THE ATTACKER OPERATED, HOW SUCCESS WAS MEASURED, AND WHAT SUPPORTS PHYSICAL OR EMBODIED RELEVANCE.

Claim element	What to specify	Why it changes interpretation	Common failure mode
Threat model	Knowledge level, access channel, privilege level, timing, query budget, physical placement, tool privileges, training or inference access	Makes ASR comparable across white-box, black-box, physical, human-mediated, and supply-chain settings	ASR without query budget, access channel, or privilege boundary.
Evidence stratum	Direct VLA, embodied-adjacent, or contextual evidence	Prevents adjacent LLM-agent, sensor, or robot-security results from being overclaimed as VLA-validated attacks	Calling a VLM jailbreak or ROS exploit a VLA attack without testing a VLA policy.
Targeted component	Perception, grounding, planner, action decoder, skill API, controller, memory, simulator, human interface	Identifies the intermediate representation or decision changed by the attack	Calling all failures “jailbreaks” without locating the attacked stage.
Attack objective	Untargeted failure, targeted action, unsafe action, freezing, goal hijack, privacy leak, unauthorized tool use, persistence	Separates task degradation from security-relevant consequences	Treating any lower task success as attack success.
ASR denominator	Frame, prompt, episode, triggered episode, trajectory, query, physical trial, or membership candidate	Determines whether success rates can be compared or aggregated	Pooling gripper-release ASR with task-failure ASR or MIA AUC.
Validation tier	Digital input-only, real-sensor input, simulation rollout, HIL, real-robot actuation, human-proximate	Prevents simulator, sensor-only, or image-level results from being overclaimed	Describing a photographed patch as real-robot actuation.
Transferability	Across prompts, models, action representations, embodiments, sensors, environments, and controllers	Determines whether the attack is substrate-specific or system-level	One model and one scene treated as general VLA risk.
Stealth and cost	Perturbation visibility, physical plausibility, attack duration, number of interventions, compute and queries	Connects attack success to deployability and detectability	Ignoring that the attack requires conspicuous or repeated intervention.
Persistence and recovery	Whether the effect survives new observations, correction, memory reset, user intervention, or safe stop	Captures closed-loop and multi-session attacks	Ending evaluation at the first wrong action.
Robot-control trace	Action representation, chunk horizon, controller frequency, safety filter, recovery policy, pre/post-controller traces	Connects model-level attack success to robotics evidence	Only decoded actions are shown when the controller clips or rejects them.
Consequence severity	Safe stop, task failure, wrong-object action, safety-envelope violation, privacy/authorization violation, human-proximate harm	Distinguishes robustness failure from security impact	Collapsing all failed episodes into a single ASR.
Defense awareness	Whether attacks are adaptive to filters, prompt hierarchy, tool permissions, shields, monitors, or human approval	Prevents evaluation from measuring only obsolete non-adaptive attacks	Evaluating a defense only against attacks designed before the defense.

and $s_{\min}$ be the minimum signed safety margin along the rollout. Useful controller-aware metrics include

$$\text{ASR}_{O|r} = \frac{\sum_i \mathbf{1}\{\kappa_O(\tau_i^\delta) = 1 \wedge r(\tau_i^\delta) = 1\}}{\sum_i \mathbf{1}\{r(\tau_i^\delta) = 1\}},$$

$$\text{CRR} = \frac{1}{N} \sum_i \mathbf{1}\{q(\tau_i^\delta) = 1\}, \quad \Delta_{\text{ctrl}} = d_{\text{post}} - d_{\text{pre}}.$$

(5)

Here $\text{ASR}_{O|r}$ is recovery-conditioned ASR, CRR is controller rejection rate, and Δ_{ctrl} measures whether the controller attenuates or amplifies the model-level deviation. Time-to-violation and $s_{\min}$ are especially important for human-proximate robots: two attacks with the same ASR may differ sharply if one violates the safety envelope after one control cycle and another remains far from contact.

B. Action Representation Changes the Claim

The same attack can mean different things under different action representations. For autoregressive action-token policies, the relevant question is whether the first changed token propagates through token-to-control mapping and subsequent tokens. For parallel action chunks, the key question is how much open-loop exposure exists before new observations can correct the command. For waypoint or trajectory policies,

displacement, curvature, workspace boundary crossing, and recovery latency matter more than token accuracy. For diffusion or flow policies, the claim depends on whether the target is the conditional representation, denoising path, velocity field, sampler, trajectory distribution, or final controller command.

The propagation mechanism differs by representation. In autoregressive action-token policies, an early token error can be amplified by the detokenizer and by subsequent conditional token generation; the relevant variables are action frequency, token bin width, and whether gripper and pose tokens are coupled. In parallel action-chunk policies, the attack may corrupt a multi-step command before new observations arrive; chunk horizon, temporal ensemble, and replan rate determine the open-loop exposure. In waypoint and trajectory policies, the attack is geometric: displacement, curvature, workspace boundary crossing, and tracking-controller error are more informative than token accuracy. In diffusion or flow policies, the target may be the conditional representation, denoising path, velocity field, sampler, or final trajectory distribution; small distributional changes can be smoothed by sampling but can also create plausible near-boundary trajectories. In skill/API policies, the safety problem is often authorization and precondition bypass: selecting a valid but unauthorized skill can be more consequential than perturbing a low-level motor command.

The physical controller must be part of the claim. A high-level action may be rejected, clipped, smoothed, delayed, or transformed by a low-level controller, safety filter, control barrier function, recovery policy, or emergency stop. If only simulator actions are shown, claims about collision/contact risk or human safety should remain tentative. If pre- and post-controller traces are available, they reveal whether the attack is attenuated, amplified, or converted into a different consequence.

C. Claim Checks by Attack Family

Different attack families need different claim checks. For *instruction-control attacks*, the key issue is whether a language change actually causes action divergence under a stated query or edit budget. RoboGCG-style control-authority claims require a different denominator from SABER-style task success reduction, Q-DIG-style failure discovery, or trajectory-redirection claims [17], [20], [31], [34]. A useful result ties together the instruction pair, target policy, rollout seed, attack budget, action divergence, consequence, and recovery outcome.

For *visual and physical attacks*, the key issue is where the perturbation lives and what representation it changes: digital noise, localized patch, printed patch, object texture, scene object, or sensor-path interference; action loss, embedding loss, attention loss, semantic misalignment, or trajectory loss. This distinction separates action-space patch attacks from EDPA/ADVLA-style feature attacks, UPA-RFAS/VLA-Hijack-style transfer attacks, Tex3D-style object-surface attacks, and partially observable patch attacks [18], [27]–[29], [35]–[37].

For *backdoor and supply-chain attacks*, the key issue is whether the trigger survives realistic adaptation while preserving clean utility. Triggered metrics should separate failure, goal completion, primitive induction, action freezing, and chunk drift [21]–[26]. A backdoor with poor benign performance is easier to detect and is less informative about realistic supply-chain risk.

For *reasoning and privacy attacks*, the metric is not ordinary task ASR. Reasoning attacks should name the attacked reasoning object, such as natural-language CoT, object entity references, bounding-box traces, subgoal plans, or driving rationales, and then measure whether the changed reasoning causally changes action [30], [38], [39]. Privacy attacks should report the membership unit, access regime, candidate distribution, and confidence metric, because sample-level and trajectory-level membership inference are not comparable to physical task failure [32], [33].

D. Representation Context

VLA attacks inherit representation assumptions from VLMs, action policies, and adversarial ML. Vision-language backbones such as CLIP, Flamingo, BLIP-2, LLaVA, MiniGPT-4, and InstructBLIP explain why embedding, attention, and cross-modal alignment are plausible VLA targets [119]–[124]. Query-efficient and transfer-oriented adversarial methods help design black-box evaluation [125]–[133]. These

works explain mechanisms; a VLA claim still requires a VLA rollout, action representation, or VLA privacy outcome.

E. Physical Validation

Physical validation determines how far a claim can travel. Digital input-only tests are useful for isolating mechanisms such as attention, likelihood, embeddings, prompt transfer, or membership features. Real-sensor input shows that a perturbation can be captured by a real camera, microphone, LiDAR, or robot sensor, but does not necessarily show robot consequence. Simulation rollout measures closed-loop behavior at scale. Hardware-in-the-loop exposes real sensors, controller timing, middleware, or safety filters while limiting hazard. Real-robot actuation shows physical execution. Human-proximate validation is required before making claims about people inside the safety boundary. Hybrid attacks should name both the delivery channel and the consequence setting, for example digital prompt with real-robot actuation, or physical patch with simulation-only consequence.

F. Benchmark Claims

Benchmarks must be matched to claims. LIBERO, CALVIN, RLBench, ManiSkill, RoboCasa, and DROID-derived settings can support manipulation and rollout comparisons, but they do not by themselves establish physical safety. AgentDojo, ToolEmu, InjecAgent, and RAG benchmarks support source-boundary and tool-risk analogies, not robot consequences. Embodied jailbreak benchmarks support action-boundary evaluation for planners and LLM-controlled robots, but direct VLA claims require VLA policies or VLA-enabled robot systems. A credible attack benchmark needs paired benign/adversarial tasks, threat assumptions, typed ASR, severity labels, transfer tests, physical validation, and recovery metrics.

G. Adaptive and Responsible Evaluation

Guardrails change the attack objective. Prompt hierarchy turns injection into source-boundary confusion; memory hygiene turns poisoning into persistence; tool permissions turn misuse into argument selection or authorization-boundary crossing; safety filters turn direct collision objectives into near-boundary, semantic misuse, or recovery-exhaustion objectives. Attack papers should state whether the attacker knows the guardrail, receives rejection feedback, adapts to it, or acts only through the physical environment.

Because VLA attacks may transfer to real robots, papers should publish sanitized prompts, aggregate results, threat models, and defense-relevant analysis while withholding reusable high-risk payloads or hardware exploit recipes when needed. Responsible disclosure is part of evaluation quality, not an afterthought.

VI. RESEARCH ROADMAP AND LIMITS

A. Physically Grounded Attacks

Physical grounding is essential because VLA attacks ultimately claim effects on robot motion, tool use, environment

TABLE VI

VALIDITY OF COMMON TESTBEDS FOR VLA ATTACK CLAIMS. A BENCHMARK SUPPORTS ONLY THE EVIDENCE LEVEL IT ACTUALLY MEASURES.

Benchmark/framework	Type	What it supports	Invalid claim if used alone	Attack overlay needed
AI2-THOR / ALFRED / Capability Habitat [134]–[136]		Grounding and household/navigation planning	Physical safety or controller-level attack success	Adversarial goals, source conflicts, recovery labels.
RLBench / CALVIN / Capability LIBERO / ManiSkill [47], [114]–[116]		Manipulation trajectories and language-conditioned tasks	Security claims from task success alone	Targeted ASR, trajectory deviation, paired benign/adversarial tasks.
RoboCasa / DROID-derived data [86], [117]	Data/testbed	Scene diversity and data provenance questions	Poisoning or sim-to-real claims without attack labels	Provenance checks, poisoned-demo splits, validation tiers.
ToolEmu / AgentDojo / In-jecAgent [52], [92], [93]	Red-team	Tool misuse and indirect prompt injection	Direct VLA or physical consequence claims	Robot skill permissions, physical-action proxy, latency.
SafeAgentBench / EARBench [82], [83]	Safety	Hazard/risk-aware planning	Low-level VLA action or hardware safety	Action traces, severity labels, adaptive attacks.
BadRobot / RoboPAIR / RoboJailBench [40], [41], [43]	Attack/red-team	Embodied jailbreak goals and security-utility pairs	Cross-platform VLA generality without more models	Paired benign goals, physical constraints, model transfer.
RoboGCG / VLA-Fool / SABER [17], [19], [20]	Direct VLA attack	Prompt, multimodal, and black-box instruction objectives	General VLA security claims from narrow model suites	Cross-policy transfer, query budgets, physical rollouts.
Patch/physical VLA attacks [18], [27]–[29]	Direct VLA attack	Visual embeddings, grounding, textures, trajectories	Universal physical safety claims from narrow scenes	Viewpoint, object, controller, and sim-to-real transfer.
FreezeVLA / DropVLA / AttackVLA [21], [24]–[26]	Direct VLA attack	Action freezing, primitives, chunks, backdoors	General action-security evidence across all VLA representations	Token/chunk/flow transfer and recovery metrics.
CHAI / ANNIE / Eva-VLA / Phantom Menace [42], [44]–[46]	Adjacent physical/sensor attack	Scene text, trajectory, physical variation, sensor path	Direct VLA attack claims without VLA policy validation	Threat checklist, recovery, cross-platform validation.
Benchmark audits [118]	Validity diagnostic	Shortcut, overfit, and statistical fragility	Attack benchmark evidence	Security splits and adversarial diagnostics.

state, or human proximity. Current work has moved beyond digital-only examples: several patch, texture, backdoor, and embodied red-team studies include simulation or real-robot demonstrations. Progress now depends less on simply adding physical trials than on making those trials more informative: calibrated cameras, lighting and viewpoint variation, object mass and friction, controller frequency, safety filters, time-to-violation, and recovery opportunities should be part of the evaluated claim.

B. HIL and Real-Robot Recovery

Recovery is the most underdeveloped robotics question. Simulation gives scale, and full physical deployment gives realism, but hardware-in-the-loop can expose real sensors, middleware timing, safety filters, controller latency, and emergency-stop logic without requiring unsafe full deployment. Future studies should show when an attack first changes representation, whether the controller clips or rejects it, whether the robot recovers after new observations, and whether human correction changes the outcome. A VLA attack that fails after one replan and a VLA attack that survives repeated recovery attempts are different risks.

C. Cross-Embodiment and Action-Representation Transfer

Transfer matters because VLA systems are defined by embodiment as much as by model name. Existing work has begun to test transfer across prompts, models, viewpoints, and simulation-to-real settings, but transfer across action representations is still thin. A prompt attack may transfer across language-conditioned token policies but fail against a flow policy; a patch may transfer across visual encoders but not camera placements; a chunk-drift backdoor may be irrelevant to a one-step replanner. Future suites should vary tokenizer, chunk length, temporal ensemble, diffusion/flow sampler, waypoint controller, robot morphology, camera geometry, and low-level safety envelope.

D. Memory, Tool, Middleware, and HRI Attacks

Deployed VLA robots will not be stateless policies. They will retrieve maps, object memories, user preferences, logs, tool outputs, and social cues. Adjacent evidence from RAG, software agents, ROS/ROS2, and HRI shows credible routes for corruption [51], [52], [54], [107], but direct VLA evidence remains thin. Future work should measure robot-level effects: unauthorized skill use, corrupted maps, persistent user preference changes, unsafe tool arguments, privacy leakage, and recovery after correction.

E. Privacy as a First-Class VLA Attack Objective

Robot actions can reveal training data, user routines, scenes, and task histories. Membership inference work shows that VLA actions and trajectories can leak training membership [32], [33]. The field now needs privacy tests beyond membership: property inference, dataset provenance, user routine leakage, private-scene memorization, and cross-session log leakage. These are direct VLA security questions even when no robot collision or task failure occurs.

F. Adaptive Attacks and Responsible Artifacts

First-generation defenses will use prompt hierarchy, source labels, retrieval filters, tool permissions, runtime monitors, action shields, control barrier functions, and human approvals. Attack research must therefore include adaptive settings: what the attacker knows, what feedback is available, whether queries are budgeted, and whether benign utility is preserved. At the same time, VLA attack artifacts can be unusually transferable to physical systems. Benchmarks should separate public scoring code and sanitized examples from high-risk payloads, physical exploit details, or deployment-specific trigger recipes.

G. Limitations of This Review

This review fixes its evidence set at the 16 June 2026 search date, so later arXiv releases, renamed project pages, and venue updates are outside scope. Google Scholar and Semantic Scholar ranking can bias discovery toward highly visible papers, while targeted-name searches can miss attacks that use different terminology. We searched English-language sources and may miss non-English reports. Several direct VLA attacks are recent preprints or workshop papers; their claims may change after peer review, and project-page or repository information is weaker evidence than archival text.

The supplementary materials document inclusion choices and supporting notes, but they do not reproduce experiments, inspect every codebase, or verify every ASR number. Some judgments remain partly subjective, especially for target representations, transfer, and validation details. The descriptive observations should therefore be read as a map of an emerging literature, not as a meta-analysis; we do not pool ASR values across attack objectives, denominators, validation tiers, or robot platforms.

VII. CONCLUSION

VLA attacks should be judged at the level of closed-loop consequences, not isolated model outputs. An adversarial effect may enter through perception, language, memory, tools, environment state, action representations, or human interaction. It may first appear as a changed object binding, plan, reasoning trace, action token, chunk, trajectory, tool call, or memory entry. The security claim becomes a robotics claim only when that change crosses the controller boundary, interacts with feedback and recovery, and reaches a task, safety, authorization, privacy, or persistence violation.

The emerging literature already shows that direct VLA systems can be attacked through patches, black-box instructions, action freezing, action-level and long-horizon backdoors, transferable and 3D object perturbations, reasoning-chain hijacking, trajectory redirection, and membership inference. Embodied-adjacent work usefully motivates risks around planners, tools, sensors, middleware, and HRI, while contextual work explains mechanisms and evaluation design. These levels should remain separate: task failure, targeted action, safety violation, privacy leakage, unauthorized skill use, persistence, and recovery failure are different predicates and should not be collapsed into one ASR.

For VLA systems, an attack claim is ultimately a claim about what survives the robot loop. Future work should therefore test whether attacks transfer across action representations, remain consequential under physical or HIL recovery protocols, and require privacy-aware metrics rather than task ASR. It should also evaluate adaptive attackers against deployed guardrails while releasing artifacts in forms that support defense without enabling misuse.

REFERENCES

- [1] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, W. Huang, Y. Chebotar, P. Sermanet, D. Duckworth, S. Levine, V. Vanhoucke, K. Hausman, M. Toussaint, K. Greff, A. Zeng, I. Mordatch, and P. Florence, “PaLM-E: An embodied multimodal language model,” in *Proceedings of the 40th International Conference on Machine Learning*, vol. 202, pp. 8469–8488, PMLR, 2023.
- [2] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, *et al.*, “RT-1: Robotics transformer for real-world control at scale,” 2022.
- [3] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choremanski, T. Ding, D. Driess, A. Dubey, C. Finn, *et al.*, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” 2023.
- [4] Y. Jiang, A. Gupta, Z. Zhang, G. Wang, Y. Dou, Y. Chen, L. Fei-Fei, A. Anandkumar, Y. Zhu, and L. Fan, “VIMA: General robot manipulation with multimodal prompts,” in *Fortieth International Conference on Machine Learning*, 2023.
- [5] A. O’Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain, *et al.*, “Open x-embodiment: Robotic learning datasets and RT-X models,” 2023.
- [6] M. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “OpenVLA: An open-source vision-language-action model,” 2024.
- [7] D. Ghosh, H. R. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, J. Luo, Y. L. Tan, L. Y. Chen, Q. Vuong, T. Xiao, P. R. Sanketi, D. Sadigh, C. Finn, and S. Levine, “Octo: An open-source generalist robot policy,” in *Proceedings of Robotics: Science and Systems*, (Delft, Netherlands), 2024.
- [8] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky, “pi0: A vision-language-action flow model for general robot control,” 2024.
- [9] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, *et al.*, “Do as i can, not as i say: Grounding language in robotic affordances,” in *Conference on Robot Learning*, 2022.
- [10] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as policies: Language model programs for embodied control,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, pp. 9493–9500, 2023.
- [11] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg, “ProgPrompt: Generating situated robot task plans using large language models,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, pp. 11523–11530, 2023.
- [12] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, “VoxPoser: Composable 3d value maps for robotic manipulation with language models,” in *Proceedings of the 7th Conference on Robot Learning*, 2023.
- [13] M. Shridhar, L. Manuelli, and D. Fox, “CLIPort: What and where pathways for robotic manipulation,” in *Proceedings of the 5th Conference on Robot Learning*, pp. 894–906, PMLR, 2022.
- [14] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Robotics: Science and Systems*, 2023.
- [15] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Proceedings of Robotics: Science and Systems*, 2023.
- [16] Q. Li, B. Yin, W. Huang, R. Liu, B. Zou, R. Yu, J. Ye, W. Yu, and X. Wang, “Vision-language-action safety: Threats, challenges, evaluations, and mechanisms,” 2026.
- [17] E. K. Jones, A. Robey, A. Zou, Z. Ravichandran, G. J. Pappas, H. Hassani, M. Fredrikson, and J. Z. Kolter, “Adversarial attacks on robotic vision language action models,” 2025.
- [18] T. Wang, C. Han, J. Liang, W. Yang, D. Liu, L. X. Zhang, Q. Wang, J. Luo, and R. Tang, “Exploring the adversarial vulnerabilities of vision-language-action models in robotics,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 6948–6958, 2025.
- [19] Y. Yan, Y. Xie, Y. Zhang, L. Lyu, H. Wang, and Y. Jin, “When alignment fails: Multimodal adversarial attacks on vision-language-action models,” 2025.
- [20] X. Wu, G. Shi, Q. Wang, Z. Li, A. S. Bedi, and D. Manocha, “SABER: A stealthy agentic black-box attack framework for vision-language-action models,” 2026.

- [21] X. Wang, J. Li, Z. Weng, Y. Wang, Y. Gao, T. Pang, C. Du, Y. Teng, Y. Wang, Z. Wu, X. Ma, and Y.-G. Jiang, "FreezeVLA: Action-freezing attacks against vision-language-action models," 2025.
- [22] X. Zhou, G. Tie, G. Zhang, H. Wang, P. Zhou, and L. Sun, "Bad-VLA: Towards backdoor attacks on vision-language-action models via objective-decoupled optimization," 2025.
- [23] Z. Zhou, Z. Xiao, H. Xu, J. Sun, D. Wang, and J. Zhang, "Goal-oriented backdoor attack against vision-language-action models via physical objects," 2025.
- [24] Z. Xu, J. Li, Y. Zhao, X. Zheng, X. Ma, and Y.-G. Jiang, "DropVLA: An action-level backdoor attack on vision-language-action models," 2025.
- [25] J. Li, Y. Zhao, X. Zheng, Z. Xu, Y. Li, X. Ma, and Y.-G. Jiang, "AttackVLA: Benchmarking adversarial and backdoor attacks on vision-language-action models," 2025.
- [26] B. Xu, Y. Shang, B. Wang, and E. Ferrara, "SilentDrift: Exploiting action chunking for stealthy backdoor attacks on vision-language-action models," 2026.
- [27] H. Xu, Y. S. Koh, S. Huang, Z. Zhou, D. Wang, J. Sakuma, and J. Zhang, "Model-agnostic adversarial attack and defense for vision-language-action models," 2025.
- [28] H. Lu, Y. Yu, Y. Yang, C. Yi, Q. Zhang, B. Shen, A. C. Kot, and X. Jiang, "When robots obey the patch: Universal transferable patch attacks on vision-language-action models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. arXiv:2511.21192.
- [29] J. Chen, S. Huang, J. Du, S. Chen, Y. Tian, M. Wei, C. Yu, and Z. Yin, "Tex3D: Objects as attack surfaces via adversarial 3d textures for vision-language-action models," 2026.
- [30] Z. Huang, W. Zhu, H. Qiu, X. Ji, and W. Xu, "TRAP: Hijacking VLA CoT-reasoning via adversarial patches," 2026.
- [31] G. Puthumanaillam, V. Dongre, P. Thangeda, H. Nayyeri, D. Hakkani-Tür, and M. Ornik, "Trajectory-level redirection attacks on vision-language-action models," 2026.
- [32] Y. Peng, M. Li, K. Xia, R. Zhang, and A. Houmansadr, "Membership inference attacks on vision-language-action models," 2026.
- [33] X. Luan, J. Liu, X. Li, Y. Bi, R. Wu, Z. Lei, and D. Wang, "VLALeaks: Membership inference attacks against vision-language-action models," 2026.
- [34] S. Srikanth, F. Liang, Y.-C. Hsu, V. Bhatt, S. Zhao, H. Chen, B. Tjanaka, M. Hwang, A. Saran, D. Seita, A. Tabrez, and S. Nikolaidis, "Red-teaming vision-language-action models via quality diversity prompt generation for robust robot policies," 2026.
- [35] N. Zhang, W. Tao, X. Xiao, Q. Sun, Y. Zheng, W. Mo, P. Wang, and N. Zhang, "Attention-guided patch-wise sparse adversarial attacks on vision-language-action models," 2025.
- [36] J. Fu, K. Jiang, J. Jia, Z. Chen, X. Chen, L. Hong, S. Gao, C. Tan, D. Yang, and W. Zhang, "VLA-Hijack: A transferable patch attack against vision-language-action models via visual proprioception hijacking," 2026.
- [37] X. Wang, M. Han, T. Hao, Y. Yang, Y.-B. Zhao, and K. Tang, "Partially observable adversarial patch attacks on vision-language-action models in robotics," *IEEE Robotics and Automation Letters*, 2026. arXiv:2606.03556.
- [38] T. D. Trinh, N. Akhtar, and B. Azam, "Altered thoughts, altered actions: Probing chain-of-thought vulnerabilities in VLA robotic manipulation," 2026.
- [39] M. Teymoorianfard, J.-P. Monteuiis, J. Petit, and A. Houmansadr, "ReasonBreak: Probing vulnerabilities in reasoning-enabled vision-language-action models for autonomous driving," 2026.
- [40] H. Zhang, C. Zhu, X. Wang, Z. Zhou, C. Yin, M. Li, L. Xue, Y. Wang, S. Hu, A. Liu, *et al.*, "Badrobot: Jailbreaking embodied LLM agents in the physical world," in *International Conference on Learning Representations*, 2025.
- [41] A. Robey, Z. Ravichandran, V. Kumar, H. Hassani, and G. J. Pappas, "Jailbreaking LLM-controlled robots," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2025. RoboPAIR; arXiv:2410.13691.
- [42] L. Burbano, D. Ortiz, Q. Sun, S. Yang, H. Tu, C. Xie, Y. Cao, and A. A. Cardenas, "CHAI: Command hijacking against embodied AI," 2025.
- [43] D. Yeke, Y. Zhou, L. Y. Lin, H. Cai, A. Bianchi, and Z. B. Celik, "Robojailbench: Benchmarking adversarial attacks and defenses in embodied robotic agents," 2026.
- [44] Y. Huang, Z. Wang, Z. Wan, Y. Tian, H. Xu, Y. Han, and Y. Gan, "Annie: Be careful of your robots," 2025.
- [45] X. Lu, J. Chen, S. Xiao, Z. Jin, Z. Chen, H. Yu, B. Qian, R. Zhou, X. Ji, and W. Xu, "Phantom menace: Exploring and enhancing the robustness of VLA models against physical sensor attacks," 2025.
- [46] H. Liu, J. Long, J. Wu, J. Hou, H. Tang, T. Jiang, W. Zhou, and W. Yao, "Eva-VLA: Evaluating vision-language-action models' robustness under real-world physical variations," 2025.
- [47] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "LIBERO: Benchmarking knowledge transfer for lifelong robot learning," in *Advances in Neural Information Processing Systems*, 2023.
- [48] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *International Conference on Learning Representations*, 2015.
- [49] T. B. Brown, D. Mané, A. Roy, M. Abadi, and J. Gilmer, "Adversarial patch," in *NeurIPS Workshop on Machine Learning and Computer Security*, 2017.
- [50] Y. Cao, C. Xiao, D. Yang, J. Fang, R. Yang, M. Liu, and B. Li, "Adversarial sensor attack on LiDAR-based perception in autonomous driving," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pp. 2267–2281, 2019.
- [51] W. Zou, R. Geng, B. Wang, and J. Jia, "PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models," in *Proceedings of the 34th USENIX Security Symposium*, 2025.
- [52] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2024.
- [53] P. Kiourti, K. Wardega, S. Jha, and W. Li, "TrojDRL: Evaluation of backdoor attacks on deep reinforcement learning," in *Proceedings of the 57th ACM/IEEE Design Automation Conference*, pp. 1–6, 2020.
- [54] G. Deng, G. Xu, Y. Zhou, T. Zhang, and Y. Liu, "On the (in)security of secure ROS2," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022.
- [55] B. Biggio, B. Nelson, and P. Laskov, "Poisoning attacks against support vector machines," in *Proceedings of the 29th International Conference on Machine Learning*, 2012.
- [56] T. Gu, B. Dolan-Gavitt, and S. Garg, "Badnets: Identifying vulnerabilities in the machine learning model supply chain," in *Neural Information Processing Systems Workshop on Machine Learning and Computer Security*, 2017.
- [57] Y. Liu, S. Ma, Y. Aafer, W.-C. Lee, J. Zhai, W. Wang, and X. Zhang, "Trojaning attack on neural networks," in *Proceedings of the Network and Distributed System Security Symposium*, 2018.
- [58] N. Carlini, M. Jagielski, C. A. Choquette-Choo, D. Paleka, W. Pearce, H. Anderson, A. Terzis, K. Thomas, and F. Tramèr, "Poisoning web-scale training datasets is practical," in *IEEE Symposium on Security and Privacy*, 2024.
- [59] L. Wang, Z. Javed, X. Wu, W. Guo, X. Xing, and D. Song, "BACK-DOORL: Backdoor attack against competitive reinforcement learning," in *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence*, pp. 3699–3705, 2021.
- [60] J. Cui, Y. Han, Y. Ma, J. Jiao, and J. Zhang, "BadRL: Sparse targeted backdoor attack against reinforcement learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, pp. 11687–11694, 2024.
- [61] E. Rathbun, A. Oprea, and C. Amato, "Adversarial inception backdoor attacks against reinforcement learning," in *Proceedings of the 42nd International Conference on Machine Learning*, pp. 51273–51296, PMLR, 2025.
- [62] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. J. Goodfellow, and R. Fergus, "Intriguing properties of neural networks," in *International Conference on Learning Representations*, 2014.
- [63] N. Papernot, P. McDaniel, S. Jha, M. Fredrikson, Z. B. Celik, and A. Swami, "The limitations of deep learning in adversarial settings," in *IEEE European Symposium on Security and Privacy*, pp. 372–387, 2016.
- [64] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *IEEE Symposium on Security and Privacy*, pp. 39–57, 2017.
- [65] A. Kurakin, I. J. Goodfellow, and S. Bengio, "Adversarial examples in the physical world," in *International Conference on Learning Representations Workshop*, 2017.
- [66] S.-M. Moosavi-Dezfooli, A. Fawzi, O. Fawzi, and P. Frossard, "Universal adversarial perturbations," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1765–1773, 2017.

- [67] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, “Towards deep learning models resistant to adversarial attacks,” in *International Conference on Learning Representations*, 2018.
- [68] A. Athalye, L. Engstrom, A. Ilyas, and K. Kwok, “Synthesizing robust adversarial examples,” in *Proceedings of the 35th International Conference on Machine Learning*, pp. 284–293, 2018.
- [69] K. Eykholt, I. Evtimov, E. Fernandes, B. Li, A. Rahmati, C. Xiao, A. Prakash, T. Kohno, and D. Song, “Robust physical-world attacks on deep learning visual classification,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1625–1634, 2018.
- [70] K. Eykholt, I. Evtimov, E. Fernandes, B. Li, A. Rahmati, F. Tramèr, A. Prakash, T. Kohno, and D. Song, “Physical adversarial examples for object detectors,” in *12th USENIX Workshop on Offensive Technologies*, 2018.
- [71] S.-T. Chen, C. Cornelius, J. Martin, and D. H. Chau, “ShapeShifter: Robust physical adversarial attack on faster R-CNN object detector,” in *Machine Learning and Knowledge Discovery in Databases*, pp. 52–68, Springer, 2018.
- [72] S. Thys, W. Van Ranst, and T. Goedemé, “Fooling automated surveillance cameras: Adversarial patches to attack person detection,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2019.
- [73] K. Xu, G. Zhang, S. Liu, Q. Fan, M. Sun, H. Chen, P.-Y. Chen, Y. Wang, and X. Lin, “Adversarial T-Shirt! evading person detectors in a physical world,” in *Computer Vision – ECCV 2020*, Springer, 2020.
- [74] C. Xiao, D. Yang, B. Li, J. Deng, and M. Liu, “MeshAdv: Adversarial meshes for visual recognition,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 6898–6907, 2019.
- [75] J. Tu, M. Ren, S. Manivasagam, M. Liang, B. Yang, R. Du, F. Cheng, and R. Urtasun, “Physically realizable adversarial examples for LiDAR object detection,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 13716–13725, 2020.
- [76] J. Sun, Y. Cao, Q. A. Chen, and Z. M. Mao, “Towards robust LiDAR-based perception in autonomous driving: General black-box adversarial sensor attack and countermeasures,” in *29th USENIX Security Symposium*, pp. 877–894, 2020.
- [77] Y. Cao, N. Wang, C. Xiao, D. Yang, J. Fang, R. Yang, Q. A. Chen, M. Liu, and B. Li, “Invisible for both camera and LiDAR: Security of multi-sensor fusion based perception in autonomous driving under physical-world attacks,” in *IEEE Symposium on Security and Privacy*, pp. 176–194, 2021.
- [78] N. Carlini, P. Mishra, T. Vaidya, Y. Zhang, M. Sherr, C. Shields, D. Wagner, and W. Zhou, “Hidden voice commands,” in *25th USENIX Security Symposium*, pp. 513–530, 2016.
- [79] G. Zhang, C. Yan, X. Ji, T. Zhang, T. Zhang, and W. Xu, “DolphinAttack: Inaudible voice commands,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pp. 103–117, 2017.
- [80] X. Yuan, Y. Chen, Y. Zhao, Y. Long, X. Liu, K. Chen, S. Zhang, H. Huang, X. Wang, and C. A. Gunter, “CommanderSong: A systematic approach for practical adversarial voice recognition,” in *27th USENIX Security Symposium*, pp. 49–64, 2018.
- [81] T. Sugawara, B. Cyr, S. Rampazzi, D. Genkin, and K. Fu, “Light commands: Laser-based audio injection attacks on voice-controllable systems,” in *29th USENIX Security Symposium*, pp. 2631–2648, 2020.
- [82] S. Yin, X. Pang, Y. Ding, M. Chen, Y. Bi, Y. Xiong, W. Huang, S. Chen, Z. Xiang, and J. Shao, “Safeagentbench: A benchmark for safe task planning of embodied LLM agents,” 2024.
- [83] Z. Zhu, B. Wu, Z. Zhang, L. Han, Q. Liu, and B. Wu, “EARBench: Towards evaluating physical risk awareness for task planning of foundation model-based embodied AI agents,” 2024.
- [84] D. Qu, H. Song, Q. Chen, Y. Yao, X. Ye, Y. Ding, Z. Wang, J. Gu, B. Zhao, D. Wang, *et al.*, “SpatialVLA: Exploring spatial representations for vision-language-action models,” *arXiv preprint arXiv:2501.15830*, 2025. Accepted to RSS 2025.
- [85] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” *arXiv preprint arXiv:2502.19645*, 2025.
- [86] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis, *et al.*, “DROID: A large-scale in-the-wild robot manipulation dataset,” in *Proceedings of Robotics: Science and Systems*, 2024.
- [87] H. R. Walke, K. Black, T. Z. Zhao, Q. Vuong, C. Zheng, P. Hansen-Estruch, A. He, V. Myers, K. Fang, C. Finn, and S. Levine, “BridgeData V2: A dataset for robot learning at scale,” 2023.
- [88] S. Dasari, F. Ebert, S. Tian, S. Nair, B. Bucher, K. Schmeckpeper, S. Singh, S. Levine, and C. Finn, “RoboNet: Large-scale multi-robot learning,” in *Conference on Robot Learning*, 2019.
- [89] S. Reed, K. Zolna, E. Parisotto, S. G. Colmenarejo, A. Novikov, G. Barth-Maron, M. Gimenez, Y. Sulsky, J. Kay, J. T. Springenberg, *et al.*, “A generalist agent,” in *Transactions on Machine Learning Research*, 2022.
- [90] S. Abdelnabi, K. Greshake, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, pp. 79–90, ACM, 2023.
- [91] J. Yi, Y. Xie, B. Zhu, K. Hines, E. Kiciman, G. Sun, X. Xie, and F. Wu, “Benchmarking and defending against indirect prompt injection attacks on large language models,” 2023.
- [92] Q. Zhan, Z. Fang, Z. Li, L. Li, and D. Kang, “InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents,” in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.
- [93] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, “Identifying the risks of LM agents with an LM-emulated sandbox,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [94] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Advances in Neural Information Processing Systems*, 2023.
- [95] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations*, 2023.
- [96] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, “Gorilla: Large language model connected with massive APIs,” 2023.
- [97] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, *et al.*, “ToolLLM: Facilitating large language models to master 16000+ real-world APIs,” in *International Conference on Learning Representations*, 2024.
- [98] S. Yao, H. Chen, J. Yang, and K. Narasimhan, “WebShop: Towards scalable real-world web interaction with grounded language agents,” in *Advances in Neural Information Processing Systems*, 2022.
- [99] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, Y. Bisk, D. Fried, U. Alon, and G. Neubig, “WebArena: A realistic web environment for building autonomous agents,” in *International Conference on Learning Representations*, 2024.
- [100] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, “ROS: An open-source robot operating system,” in *ICRA Workshop on Open Source Software*, 2009.
- [101] B. Dieber, R. White, S. Taurer, B. Breiling, G. Caiazza, H. Christensen, and A. Cortesi, “Penetration testing ROS,” in *Robot Operating System (ROS): The Complete Reference (Volume 4)*, pp. 183–225, Springer, 2020.
- [102] D. Quarta, M. Pogliani, M. Polino, F. Maggi, A. M. Zanchettin, and S. Zanero, “An experimental security analysis of an industrial robot controller,” in *IEEE Symposium on Security and Privacy*, pp. 268–286, 2017.
- [103] T. Bonaci, J. Herron, T. Yusuf, J. Yan, T. Kohno, and H. J. Chizeck, “To make a robot secure: An experimental analysis of cyber security threats against teleoperated surgical robots,” in *arXiv preprint arXiv:1504.04339*, 2015.
- [104] V. Mayoral-Vilches, U. A. Carbajo, E. Gil-Uriarte, *et al.*, “Robot cybersecurity: a review,” *International Journal of Cyber Forensics and Advanced Threat Investigations*, 2020.
- [105] J.-P. A. Yaacoub, H. N. Noura, O. Salman, and A. Chehab, “Robotics cyber security: Vulnerabilities, attacks, countermeasures, and recommendations,” *International Journal of Information Security*, vol. 21, pp. 115–158, 2022.
- [106] S. Neupane, S. Mitra, I. A. Fernandez, S. Saha, S. Mittal, J. Chen, N. Pillai, and S. Rahimi, “Security considerations in AI-robotics: A survey of current methods, challenges, and opportunities,” *IEEE Access*, vol. 12, pp. 22072–22097, 2024.
- [107] S. Booth, J. Tompkin, K. Z. Gajos, J. Waldo, H. Pfister, and R. Nagpal, “Piggybacking robots: Human-robot overtrust in university dormitory security,” in *Proceedings of the 2017 ACM/IEEE International Conference on Human-Robot Interaction*, pp. 426–434, 2017.
- [108] B. Postnikoff and I. Goldberg, “Robot social engineering: Attacking human factors with non-human actors,” in *Companion of the 2018 ACM/IEEE International Conference on Human-Robot Interaction*, pp. 313–314, 2018.

- [109] W. A. Bainbridge, J. Hart, E. S. Kim, and B. Scassellati, “The benefits of interactions with physically present robots over video-displayed agents,” in *International Journal of Social Robotics*, vol. 3, pp. 41–52, 2011.
- [110] S. P. Saunderson and G. Nejat, “Persuasive robots should avoid authority: The effects of formal and real authority on persuasion in human-robot interaction,” *Science Robotics*, vol. 6, no. 58, p. eabd5186, 2021.
- [111] Z. Zhong, Z. Huang, A. Wettig, and D. Chen, “Poisoning retrieval corpora by injecting adversarial passages,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 13764–13775, 2023.
- [112] G. Deng, Y. Liu, K. Wang, Y. Li, T. Zhang, and Y. Liu, “PANDORA: Jailbreak GPTs by retrieval augmented generation poisoning,” 2024.
- [113] S. Cho, S. Jeong, J. Seo, T. Hwang, and J. C. Park, “Typos that broke the RAG’s back: Genetic attack on RAG pipeline by simulating documents in the wild via low-level perturbations,” in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024.
- [114] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison, “RLBench: The robot learning benchmark and learning environment,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3019–3026, 2020.
- [115] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard, “CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks,” *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7327–7334, 2022.
- [116] T. Mu, Z. Ling, F. Xiang, D. Yang, X. Li, S. Tao, Z. Huang, Z. Jia, and H. Su, “ManiSkill: Generalizable manipulation skill benchmark with large-scale demonstrations,” in *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.
- [117] S. Nasiriany, A. Maddukuri, L. Zhang, A. Parikh, A. Lo, A. Joshi, A. Mandalekar, and Y. Zhu, “RoboCasa: Large-scale simulation of household tasks for generalist robots,” in *Proceedings of Robotics: Science and Systems*, (Delft, Netherlands), 2024.
- [118] T. Jiang, X. Tan, S. Wheeler, L. Sun, T. W. Ayalew, and M. Walter, “What are we actually benchmarking in robot manipulation?” 2026.
- [119] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, *et al.*, “Learning transferable visual models from natural language supervision,” in *Proceedings of the 38th International Conference on Machine Learning*, PMLR, 2021.
- [120] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, *et al.*, “Flamingo: A visual language model for few-shot learning,” in *Advances in Neural Information Processing Systems*, 2022.
- [121] J. Li, D. Li, S. Savarese, and S. Hoi, “BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models,” in *Proceedings of the 40th International Conference on Machine Learning*, pp. 19730–19742, PMLR, 2023.
- [122] H. Liu, C. Li, Q. Wu, and Y. J. Lee, “Visual instruction tuning,” in *Advances in Neural Information Processing Systems*, 2023.
- [123] D. Zhu, J. Chen, X. Shen, X. Li, and M. Elhoseiny, “MiniGPT-4: Enhancing vision-language understanding with advanced large language models,” 2023.
- [124] W. Dai, J. Li, D. Li, A. M. H. Tiong, J. Zhao, W. Wang, B. Li, P. Fung, and S. Hoi, “InstructBLIP: Towards general-purpose vision-language models with instruction tuning,” in *Advances in Neural Information Processing Systems*, 2023.
- [125] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z. B. Celik, and A. Swami, “Practical black-box attacks against machine learning,” in *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security*, pp. 506–519, 2017.
- [126] F. Tramèr, A. Kurakin, N. Papernot, I. Goodfellow, D. Boneh, and P. McDaniel, “Ensemble adversarial training: Attacks and defenses,” in *International Conference on Learning Representations*, 2018.
- [127] P.-Y. Chen, H. Zhang, Y. Sharma, J. Yi, and C.-J. Hsieh, “ZOO: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models,” in *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security*, pp. 15–26, 2017.
- [128] W. Brendel, J. Rauber, and M. Bethge, “Decision-based adversarial attacks: Reliable attacks against black-box machine learning models,” in *International Conference on Learning Representations*, 2018.
- [129] A. Ilyas, L. Engstrom, A. Athalye, and J. Lin, “Black-box adversarial attacks with limited queries and information,” in *Proceedings of the 35th International Conference on Machine Learning*, pp. 2137–2146, PMLR, 2018.
- [130] Y. Li, L. Li, L. Wang, T. Zhang, and B. Gong, “NATTACK: Learning the distributions of adversarial examples for an improved black-box attack on deep neural networks,” in *Proceedings of the 36th International Conference on Machine Learning*, pp. 3866–3876, PMLR, 2019.
- [131] M. Andriushchenko, F. Croce, N. Flammarion, and M. Hein, “Square attack: A query-efficient black-box adversarial attack via random search,” in *Computer Vision – ECCV 2020*, pp. 484–501, Springer, 2020.
- [132] F. Croce and M. Hein, “Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks,” in *Proceedings of the 37th International Conference on Machine Learning*, pp. 2206–2216, PMLR, 2020.
- [133] Y. Dong, F. Liao, T. Pang, H. Su, J. Zhu, X. Hu, and J. Li, “Boosting adversarial attacks with momentum,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 9185–9193, 2018.
- [134] E. Kolve, R. Mottaghi, W. Han, E. VanderBilt, L. Weihs, A. Herrasti, D. Gordon, Y. Zhu, A. Gupta, and A. Farhadi, “AI2-THOR: An interactive 3d environment for visual AI,” 2017.
- [135] M. Shridhar, J. Thomason, D. Gordon, Y. Bisk, W. Han, R. Mottaghi, L. Zettlemoyer, and D. Fox, “ALFRED: A benchmark for interpreting grounded instructions for everyday tasks,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 10740–10749, 2020.
- [136] M. Savva, A. Kadian, O. Maksymets, Y. Zhao, E. Wijmans, B. Jain, J. Straub, J. Liu, V. Koltun, J. Malik, *et al.*, “Habitat: A platform for embodied AI research,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 9339–9347, 2019.